Teacher notes: Introduction to Python and pandas

Afeez Sule · DSN FUOYE · Class 1

The purpose of this class

Help beginners understand what Python is, run code in a Jupyter notebook inside VS Code, and answer simple questions about an orders table using pandas. The main practical library is pandas. The ipykernel package supports notebook execution. Plotting is reserved for another class.

The orders dataset comes from Tech With Tim's PanadasTutorial. We use the supplied CSV with newly written notebooks and teaching notes. It has 40 order records and nine columns. Keep the source file unchanged. Exports go in outputs/.

Before students arrive

Open the repository folder in desktop VS Code, then open the notebooks. Install the Microsoft Python and Jupyter extensions. Create .venv, install requirements.txt, and select that environment as the notebook kernel. Run 00_setup_check.ipynb. Test the projector and the size of displayed code.

Share the README setup instructions before the session. Clone the repository on your own laptop and test it from the same folder structure students will use. Do not rely on an old standalone notebook from the earlier student-score lesson; this class uses the orders repo.

Rehearse your opening aloud. Keep water and this guide nearby. Speak slowly enough to point at each unfamiliar symbol. After a question, wait five seconds. You do not need to fill every silence.

A realistic one-hour route

Time	Focus	Material
0-7 min	What data science and Python mean; the class question	Opening below
7-25 min	Python basics	01_python_basics.ipynb
25-35 min	pandas, CSV, DataFrame, inspection	Notebook 02, sections 1-2
35-48 min	Select columns/rows and filter	Notebook 02, sections 3-7
48-55 min	Count and save	Notebook 02, sections 8-9
55-60 min	Small challenge and recap	Practice B or C

Installation should be completed before this hour. If learners need installation help live, allow a separate setup period. If the room needs more Python practice, shorten the pandas exercise rather than rushing. Notebook 03 contains all the additional topics from the preparation screenshots. Treat it as extension material or a second session, not another requirement to finish in the hour.

Your opening

Say: Good afternoon, everyone. I’m Afeez, a Statistics student here at FUOYE. Today we’ll learn how to give a computer simple instructions with Python, then use pandas to explore a table of orders. You don’t need programming experience. We’ll explain the tools before using them and try small examples together.

Ask: If a shop gave you a table of its orders, what would you want to find out? Take two answers. Offer examples if needed: which products appear, where orders come from, or which orders contain many units.

Explain: Data are recorded observations or facts. Data science combines statistics, computing and subject knowledge to learn from data. A result needs context. A count of orders from a country is not a count of all customers in that country.

Python and the tools around it

Python is a general-purpose programming language. Code is the instructions we write; an interpreter executes them. The editor is where we write. VS Code is the editor, not the language. A notebook combines explanation, code cells and output. Its kernel is the running Python environment.

A library supplies reusable code for particular tasks. pandas helps us work with tables. A framework supplies a broader structure and conventions for an application. A toolbox is a useful analogy for a library; scaffolding is a useful analogy for a framework. Mention frameworks briefly if asked, then return to the lesson. We do not install web frameworks for this class.

A virtual environment gives the project a separate set of packages. pip installs packages. Installing pandas makes it available in that environment; importing it makes it available in the current Python program. ipykernel enables the environment to execute notebook cells in VS Code.

Setup instructions to demonstrate

Use README.md as the student-facing setup guide. The standard Windows instructions below are for Command Prompt. PowerShell users can use the README's direct-interpreter alternative without changing execution policy.

```
git clone https://github.com/Cyberpunk738/Data-Science-tutorial.git
cd Data-Science-tutorial
py -3 -m venv .venv
.venv\Scripts\activate.bat
python -m pip install -r requirements.txt
python scripts/check_setup.py
```

For macOS/Linux, use python3 -m venv .venv and source .venv/bin/activate, then the same pip/check commands. If Git or Python was just installed, reopen the terminal before retrying.

In VS Code: open notebooks/00_setup_check.ipynb, choose Select Kernel, then Python Environments, and select this repository's .venv. Run its cells. The ready message must say 40 orders and 9 columns. The package installation terminal and the notebook must use the same Python environment.

How to explain Python basics

print, quotes and parentheses

```
print("Hello, data science!")
print(2 + 3)
```

Say: print is a function, a named tool that performs a task. Its parentheses hold inputs. Quotes mark text. The second line displays 5 because Python calculates the expression. Ask learners to replace the greeting with their names.

Variables and assignment

```
quantity = 3
print(quantity)
print("quantity")
```

Say: quantity is a name referring to the number 3. A single equals sign assigns a value. The quoted word is literal text, so the third line prints quantity. It does not look up the variable. Ask learners to predict both outputs before running.

Types

Type	Example	Plain explanation
int	3	Whole number
float	45.0	Number represented with a decimal point
str	"Alice Wong"	Text
bool	True	True or False

True and False have capital first letters and no quotes. "Yes" and "No" in the CSV are strings, not automatically booleans. Show type() once. Explain that 3 and "3" may look similar on screen but support different operations.

Calculations and lists

Use +, -, * and /. Parentheses can group arithmetic. Keep the arithmetic exercise separate from the meaning of the CSV Price column, because the source does not specify currency or whether Price is per unit.

```
countries = ["USA", "Canada", "UAE"]
print(countries[0])
print(len(countries))
```

Say: a list keeps several values together. Commas separate them. The first position is zero. len counts items. Predict countries[1]: Canada.

Comparisons and if

```
quantity = 20
print(quantity > 20)
print(quantity >= 20)
print(quantity != 20)
```

These give False, True, False. Explain >, >=, <, <=, == and != in plain words. Distinguish = for assignment from == for comparison.

```
quantity = 3
if quantity >= 2:
    print("Multiple units")
else:
    print("One unit or fewer")
```

A colon starts each block. Four spaces indent the instructions belonging to it. Change quantity to 1 and ask which branch runs. Avoid introducing loops or nested decisions in this class.

A brief dictionary preview

```
order = {"CustomerName": "Alice Wong", "Quantity": 3}
print(order["CustomerName"])
```

Curly braces hold a dictionary. A colon pairs a key with a value. This prepares students to recognise labelled data without requiring them to build the whole CSV by hand.

Main pandas lesson: say it before you type it

1. pandas, pd, df and CSV

Say: pandas is a Python library for analysing tables. We will give it the short name pd. A DataFrame is a table with labelled rows and columns. df is the variable name we choose for it.

```
import pandas as pd
df = pd.read_csv(DATA_FILE)
df.head()
```

DATA_FILE comes from the preparation helper cell and points to data/orders.csv. If teaching a simple relative path with the kernel definitely at the repository root, the equivalent is pd.read_csv("data/orders.csv"). The supplied helper works from the notebooks folder too. Explain the practical purpose of the helper, but do not turn pathlib into a separate library lesson.

A CSV is a text file with comma-separated fields. pd.read_csv reads it. head() shows the first five rows. Notebook cells display their last expression automatically. In the .py demo, print displays results explicitly.

2. The actual columns and their meaning

The exact labels are OrderID, CustomerName, Product, Category, Quantity, Price, OrderDate, Shipped and Country. One row is an order record. Price has no specified currency. OrderDate is stored as text by this simple read_csv call; we do not parse dates in this lesson.

```
df.columns
df.shape
len(df)
```

Say: columns is an attribute, so we read it without parentheses. shape gives (40, 9). len(df) returns 40 rows. head() is a method, so we call it with parentheses. This helps explain why df.columns() raises an error.

3. One column and its length

```
df["Country"]
len(df["Country"])
```

Selecting one column gives a Series. The length is 40 entries, including any missing entries if there were any. It is not the number of distinct countries. Use df["Country"].nunique() for distinct non-missing labels, which is 28 here. Column spelling and capitalisation must match exactly.

4. Several columns

```
df[["CustomerName", "Country"]]
```

Say: the inner brackets make a list of column names. The outer brackets select them from the DataFrame. Have learners select Product, Category and Quantity as practice A.

5. Integer-position rows with iloc

```
df.iloc[0]
df.iloc[0:3]
```

Say: iloc means selection by integer position. It uses square brackets. Position zero is the first row. The slice returns positions 0, 1 and 2; it stops before 3. OrderID 1001 is a field value, not automatically the index label. loc uses labels or conditions, while iloc uses integer positions. Avoid saying iloc is a function called with parentheses.

6. One filter

```
electronics = df["Category"] == "Electronics"
print(electronics.head())
df[electronics]
```

First produce one True/False result for each row. Then keep the True rows. Show the compact form after explaining the mask:

```
df[df["Category"] == "Electronics"]
```

Category is singular and starts with a capital C. Electronics also starts with a capital E. This yields 18 orders. Comparing == filters; assigning with = would be a different operation.

7. Quantity comparisons

```
df[df["Quantity"] > 20]
df[df["Quantity"] >= 20]
df[df["Quantity"] >= 2]
df[df["Quantity"] <= 2]
df[df["Country"] != "USA"]
```

The first result is Alex Johnson's order for 100 units. The second includes Ahmed Hassan's 20-unit order too. >= and <= include equality. != excludes matching values. Ask learners to predict the boundary behaviour before executing.

8. Count labels

```
df["Country"].value_counts()
```

USA, South Korea and Japan each appear on three order records. Counts refer to rows/orders, not distinct customers. Tied labels may appear in a different order between pandas versions; their counts are the important part.

9. Save the selected rows

```
usa_orders = df[df["Country"] == "USA"]
usa_orders.to_csv(OUTPUT_DIR / "usa_orders.csv", index=False)
```

OUTPUT_DIR is the outputs folder created by the preparation cell. to_csv writes a file. index=False prevents pandas from writing the row index as another CSV field. False is a Python boolean with a capital F. Saving a new file is different from merely changing a DataFrame in memory. The original data/orders.csv remains unchanged.

Stop the required lesson here. Practice B has three USA orders; practice C has eight Stationery orders. Use teacher/solutions.ipynb only after students try.

Additional examples from your preparation screenshots

All of these are in notebook 03. They run against the same original dataset, so the examples do not depend on names or columns that are absent from the CSV.

AND and OR filters

```
df[(df["Category"] == "Electronics") & (df["Country"] == "USA")]
df[(df["Category"] == "Electronics") | (df["Country"] == "USA")]
```

Say: & means both conditions must match; | means at least one may match. Each condition needs its own parentheses. Use these operators for pandas Series instead of Python's and/or keywords. The AND result has one row, John Smith's order. The OR result has 20 rows, including rows satisfying both conditions once.

Text starts and ends

```
df[df["CustomerName"].str.startswith("A", na=False)]
df[df["CustomerName"].str.endswith("son", na=False)]
```

The methods are startswith and endswith. .str accesses text methods for the Series. Parentheses pass the pattern to the method. na=False means a missing name should not match. Matching is case-sensitive. There are seven names beginning with A and five ending with son in this snapshot.

Locate a customer, then edit one value

```
df.loc[df["CustomerName"] == "Anna Ivanova", ["CustomerName", "Product"]]

edited_df = df.copy()
edited_df.loc[edited_df["CustomerName"] == "Anna Ivanova", "Product"] = "Desk Timer"
```

The screenshot's Avanova is not an exact CustomerName in the CSV. Anna Ivanova exists, so we use her row. Its original product is Desk Clock; Desk Timer is an illustrative replacement for the exercise. A condition belongs inside the first loc slot. The second slot names the column. Assignment occurs outside the closing bracket. This changes a copy, not the source file.

A name can appear on several orders in a real dataset, so this pattern changes every matching row. To target a unique order, filter OrderID instead. Avoid chained assignment such as edited_df[mask]["Product"] = ...

Replace every matching country value

```
edited_df.loc[edited_df["Country"] == "USA", "Country"] = "United States"
```

Say: the condition selects all USA rows. The second Country tells pandas which column to change. There are three matching rows. This replaces values; it does not rename the Country column or change other countries.

Rename a column

```
edited_df = edited_df.rename(columns={"OrderID": "Order ID"})
```

Use columns=, plural. The dictionary maps old column label to new label. This form returns a new table, so we assign it. The alternative is edited_df.rename(columns={"OrderID": "Order ID"}, inplace=True). That form modifies edited_df and returns None; do not assign its return value to edited_df.

Count and save the edited table

```
edited_df["Country"].value_counts()
edited_df.to_csv(OUTPUT_DIR / "orders_edited.csv", index=False)
```

After the replacement, the count label is United States rather than USA. After renaming, the identifier column is Order ID rather than OrderID. Rerun the original read_csv cell to restore the untouched data in memory. Rerun copy/update/rename cells in sequence to reproduce the edited export.

Corrections to keep beside you

Preparation shorthand	Correct form or explanation
df.columns()	df.columns, an attribute
country / countries / categories / quantities	Country / Country / Category / Quantity in this CSV
Multiple column names	df[["CustomerName", "Country"]]
df.iloc()	df.iloc[0], integer-position selection
Two conditions without grouping	Parenthesise each condition, then combine with & or pipe
startwith / endwith	startswith / endswith
CustomerName == inside a quoted label	df["CustomerName"] == "Anna Ivanova"
column= in rename	columns=
index=false	index=False
len(Country) means country count	It counts entries; nunique() counts distinct labels

Expected answers

Question	Result
Table shape	40 rows, 9 columns
Entries in Country	40
Distinct country labels	28
Electronics orders	18
USA orders	3
Stationery orders	8
Electronics AND USA	1
Electronics OR USA	20
Quantity > 20	Alex Johnson, 100 units
Quantity >= 20	Ahmed Hassan, 20; Alex Johnson, 100
Customer names starting A	7
Customer names ending son	5
Anna Ivanova's source product	Desk Clock
USA labels replaced in edited copy	3

Beginner questions and recovery lines

Question or error	How to explain or respond
What is pd?	The short alias we choose for pandas.
What is df?	The variable name we choose for the DataFrame.
What is a method?	A function associated with an object, such as df.head().
Why does df.columns have no ()?	We read an attribute; we are not calling a method.
Why both lists and Series?	A list is a Python structure; a Series adds pandas labels and operations.
Where is my data saved?	A DataFrame is in memory until exported; the supplied CSV is on disk.
KeyError	Check the exact column label and case.
NameError	Check spelling and run the earlier defining cells.
ModuleNotFoundError	Check the selected kernel, then install requirements into that environment.
File not found	Open the repo folder and rerun the path helper; keep data/orders.csv in place.
Ambiguous truth value	Check each condition's parentheses and the & / pipe operator.
I cannot answer a question	Say: I’m not certain. I’ll verify it before giving you an answer. Record it.
I lose my place	Say: Let us return to our question about the orders table.

Closing

Ask: what does pd mean? What is the difference between selecting one column and filtering rows? How would you save only USA orders?

Thank the class and direct them to notebooks 01 and 02 for practice. Explain that notebook 03 is additional practice and that the solution notebook is available after they try. Encourage learners to restart a kernel and run their work from the top to check that it is reproducible.

Sources

• Dataset reference: https://github.com/techwithtim/PanadasTutorial

• Dataset snapshot and field notes: data/README.md in this repository

• VS Code notebook setup: https://code.visualstudio.com/docs/datascience/jupyter-notebooks

• VS Code kernel selection: https://code.visualstudio.com/docs/datascience/jupyter-kernel-management

• pandas selection: https://pandas.pydata.org/docs/user_guide/indexing.html

• pandas startswith: https://pandas.pydata.org/docs/reference/api/pandas.Series.str.startswith.html

• pandas CSV export: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html